Chapitre VI:

Cassandra

Modèle de données

Gestion des données et Objets

- **Deux interfaces pour gérer les objets/donnée**
  - Cassandra CLI (Commande Line Interface)
  - CQL (Cassandra Query Language)

- **CLI**
  - Interface originelle conçue pour créer des objets, entrées et manipuler les données

- **CQL**
  - Cassandra Query Language Très proche du SQL (version simplifiée pour le NoSQL)
  - Utilisée pour créer /manipuler des données en utilisant un langage proche de SQL

Notion de bases

- Structure orientée colonnes à l’image de **BigTable**
- Vocabulaire:
  - **Keyspace:** équivalent de database
  - **Column Family** (équivalent d’une table) une colonne est composée d'un nom, d'une valeur et d'un timestamp (instant de création).
    - Dans la nouvelle version du langage CQL, appelée Table
    - Schéma plus flexible et dynamique qu’une table
  - **Colonne** (équivalent d’un enregistrement)
    - Indexées par une clef
    - D’autres champs peuvent être également indexés, mais à la demande

Notion de bases

- **Modèle = relationnel + types complexes**
  - **Bases de données :** Keyspace
  - **Tables :** Table ou Column Family
  - **Lignes :** Row (valeurs simples ou complexes)
  - **Une colonne** est la plus petite unité du modèle de données de Cassandra.
  - C'est un triplet contenant « **un nom, une valeur et un timestamp ».**

- Soit **c'est du relationnel étendu en rompant la première règle de normalisation** (type atomique).

- Ou bien **chaque ligne est un document structuré** (**imbrication**)

Notion de bases

- La structure de base est la paire **(clé, valeur)**
  - **Clé :** un identifiant
  - **Valeur atomique** (entier, chaîne de caractères), ou **Valeur complexe** (dictionnaire, ensemble, liste)

- Une ligne **(row, document)** est un identifiant **(row key)** associé à un ensemble de paires (clé, valeur).

- Les lignes sont **typées** par un schéma, y compris les données imbriquées. Cassandra n'autorise pas l'insertion de données ne respectant pas le schéma.

Structure orienté colonne: La colonne

- La colonne est la plus petite unité de donnée de Cassandra. Elle est constituée du triplet :
  - **Nom**
  - **Valeur :** Maximum 2G. Elle n'est pas obligatoire.
  - **Timestamp :** Sauvegarde la mise à jour la plus récente.

- **Important:** une colonne dans cassandra n'a pas le même sens que dans un SGBDR. Il s'agit là d'un attribut d'un enregistrement.

Nom
Valeur
Timestamp

Code
M1234

Structure orientée colonnes: ligne

- Une ligne es e par une clé.
- C'est l'équiv

mat	noom	adresse
	youcef	
	mohamed	
	ahmed	

Cle :	Nom	Nom	Nom
	Youcef	Mohamed	Ahmed
	Timestamp	Timestamp	Timestamp

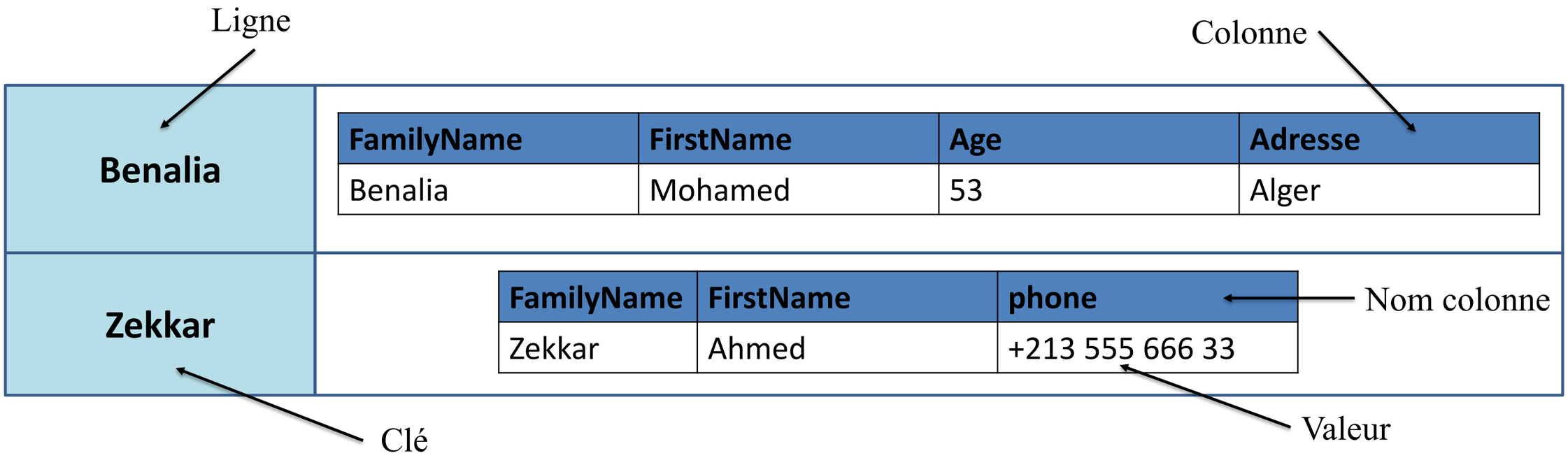
Cle	Nom	Prénom	Ville
	Benalia	Mohamed	Oran

Structure orientée colonnes

- **Le timestamp** sert à déterminer la mise à jour la plus récente.
- La taille du nom peut être contenue jusqu'à **64 KO**. La valeur quant à elle peut contenir **2 GO** de données.
- La valeur n'est pas obligatoire. Son omission peut conduire à une amélioration de performance.

Structure orientée colonnes: Ligne

- Une ligne est composée d'un ensemble de colonnes.
- Elle est identifiée par une clé. Une clé peut contenir jusqu'à 64 Ko de données et peut contenir jusqu'à deux milliards de colonnes.
- Il est possible d'utiliser des colonnes comme clé primaire.
- Les colonnes des lignes ne sont pas forcément les mêmes.



Structure orientée colonnes: notion de clé

- Les données étant réparties sur différent noeuds, il faut pouvoir la récupérer rapidement. Elle est codé sur 128bits et généralement représentée sous la forme de groupes de caractères hexadécimaux en minuscule séparés par des tirets.
  - Cette clé d'enregistrement est de type **uuid (Universal Unique IDentifier)**

- Il est possible de définir des clés primaires similaires aux SGBDR. Dans ce cas, la première colonne de la clé sera utilisé comme clé d'enregistrement (et convertit). Les suivantes ne serviront qu'à s'assurer que l'enregistrement est unique.

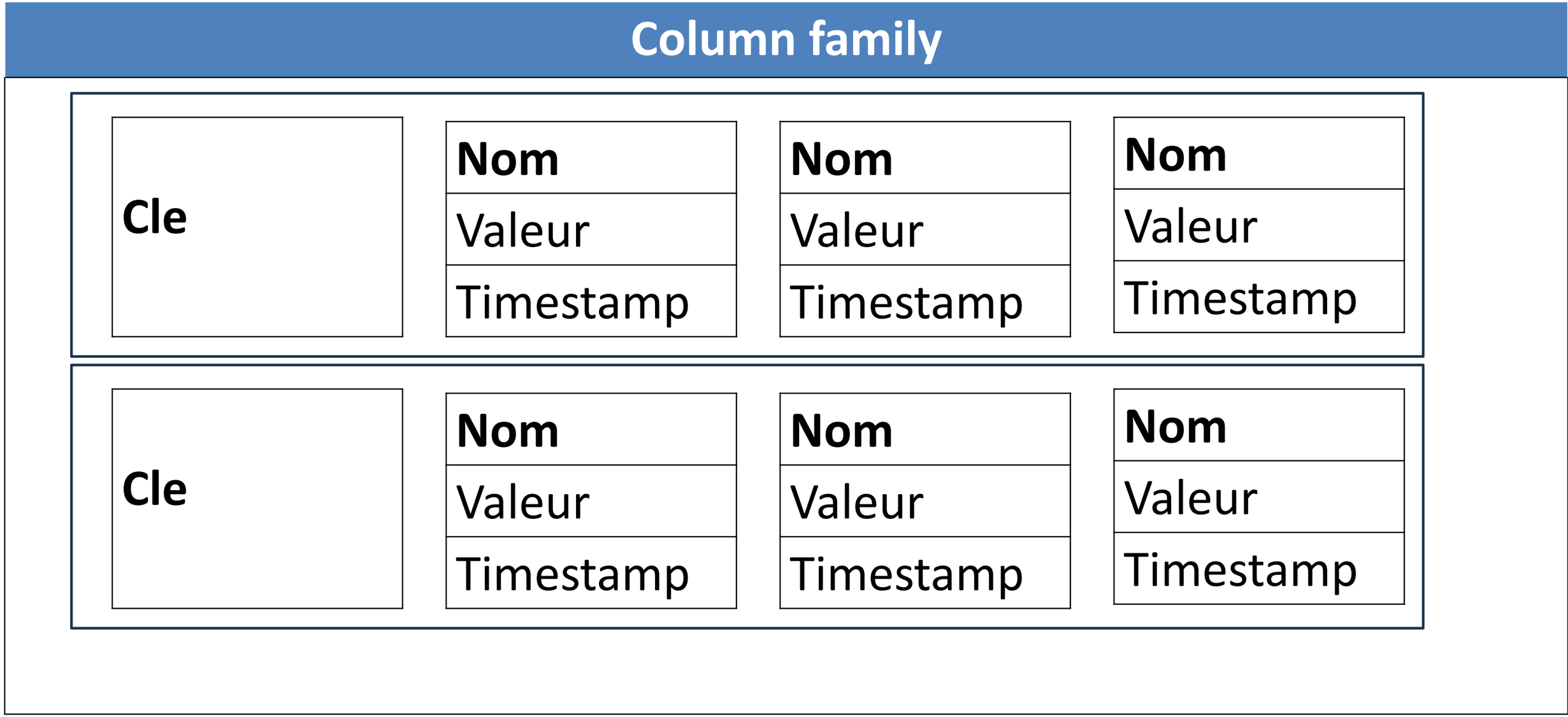
Structure orientée colonnes: *column family*

- Une famille de colonnes ou ***column family*** est un regroupement logique de lignes. Ça correspond en quelque sorte à une table relationnelle

- Lors de la définition d'une famille de colonnes, il est possible d'ajouter des informations concernant les métadonnées des colonnes.

- Ces informations peuvent être utilisées pour renseigner le nom et le type des colonnes.

Structure orientée colonnes: *column family*



Structure orientée colonnes: *column family*

- **Employes**

Cle	Nom	Prenom	fonction
	Banalia	Mohamed	Ingenieur

Cle	Nom	Age
	Slimani	55

Cle	Nom	Prenom	fonction
	Ramdani	Ali	Comptable

Structure orientée colonnes: *column family*

- **Une Column Family** est l'équivalent d'une **Table** dans un SGBDR.

- On peut d'ailleurs y ajouter des "**métadonnées**", en quelque sorte des "entêtes" de colonnes. Néanmoins, les colonnes définies ne seront pas forcément exploités lors de la création de ligne.

- Il existe deux types de familles de colonnes
  - **statique :** les colonnes sont définies lors de la création ou modification de la famille de colonnes ;
  - **dynamique :** les colonnes sont définies lors de la création ou modification d'une ligne.

Structure orientée colonnes: *column family*

- **Table:** mise en forme des données lignes/colonne (**CREATE/ DROP/ ALTER/INSERT**)
- **Colonne caractéristique** des lignes avec des types
- Au moment de l'ajout d'une ligne que vous choisirez quelles sont les colonnes à exploiter, d'où Cassandra distingue deux types de famille de colonnes:

  - **Statique :** les colonnes sont définies lors de la création ou modification de la famille de colonnes ;
  - **Dynamique:** les colonnes sont définies lors de la création ou modification d'une ligne.

Persons

Benalia

FamilyName	FirstName	Age	Adresse
Benalia	Mohamed	53	Alger

Zekkar

FamilyName	FirstName	phone
Zekkar	Ahmed	+213 555 666 33

Structure orientée colonnes: Keyspace

- **Le Keyspace (database/schema)** est un regroupement de famille de colonnes.
- Il s'agit d'une sorte de schéma si on compare au monde des bases de données relationnelles.

CassandraDEMO

Persons

Benalia

FamilyName	FirstName	Age	Adresse
Benalia	Mohamed	53	Alger

Zekkar

FamilyName	FirstName	phone
Zekkar	Ahmed	+213 555 666 33

...

Gestion des données et Objets

- **CQL**
  - Objets tels que les **keyspaces**, famille de colonnes et index sont crées, modifiés et supprimés avec les requêtes usuelles: **CREATE**, **ALTER**, et **DROP**
  - Données insérées, modifiées et supprimées avec **INSERT**, **UPDATE**, et **DELETE**
  - Données lues avec **SELECT**

- **Cependant:**
  - Ne supporte pas des opérations telles que **GROUP BY**, **ORDER BY** (sauf pour les clefs composées et ordonnées seulement selon la deuxième clef primaire)
  - Utiliser la clause **USING CONSISTENCY** pour déterminer le type de consistance forte pour chaque opération de lectures et l'écriture (Any, One, Quorum)

Création d'une base de données

- **Création d'un Keyspace**

```
CREATE KEYSPACE <keyspace_name>
WITH REPLICATION = { 'class' : '<class de la strategie>', 'replication_factor' : <nb réplication> };
```

  - **class :** Strétagie de gestion des uuid
    - La stratégie Simple Strategy convient pour les clusters locaux.
  - **replication_factor :** Nombre de noeuds sur lesquels les données doivent être répliquées

- **Création d'un Keyspace**

```
USE <keyspace_name> ;
```

- **Supprimer un keyspace: Suppression d'un keyspace et de tout son contenu**

```
DROP KEYSPACE <keyspace_name> ;
```

Affichage d’informations

- **Liste des Keyspaces existantes**

```
DESCRIBE KEYSPACES
```

- **Structured’unkeyspace: cela affiche toutes les commandes servant à le reconstruire ainsi que ses tables**

```
DESCRIBE KEYSPACE nomkeyspace;
```

- **Sélectiond’unkeyspacepourtravailler**

```
USE nomkeyspace;
```

- **Création d'une column family avec une clé primaire naturelle**
  - **Syntaxe:**

```
CREATE TABLE <table>
 {
     champs1 type,
     champs2 type,
      ....
     PRIMARY KEY(champs1)
} ;
```

- **Création d'une column family avec une clé primaire naturelle**
  - **Exemple:**

```
CREATE TABLE livre (
isbn varchar,
titre varchar,
PRIMARY KEY(isbn));
```

- **Création d'une column family avec une clé primaire artificielle**
  - **Syntaxe**

```
CREATE TABLE <table>
 {
    id              uuid,
    champs2         type,
      ....
       (id)
} ;
```

- **Création d'une column family avec une clé primaire artificielle**
  - **Exemple**

```
CREATE TABLE livre
 (
    id uuid,
     isbn varchar,
      titre varchar,
      PRIMARY KEY(id)
);
```

- Soit une table représentant des clients :

```
CREATE TABLE clients
(
    idclient        INT,    --n° du client
    departement     INT,    --n° du département,    ex:22,29,35...
    Nom             TEXT,...—coordonnées du client...
   PRIMARYKEY(...)
)
```

- On a plusieurs possibilités pour **la contrainte PRIMARYKEY**:
  - **PRIMARYKEY(idclient):** les n-uplets sont identifiés par le n°client, c'est la«rowkey»
  - **PRIMARYKEY(departement, idclient):** la clé est composite, departement sert de «clé de partition».
    - Tous les clients du même département seront sur la même machine et ils seront classés par idclient.

- ## Création d’un index secondaire
  - La clé est un index primaire.
  - Syntaxe d’ajout d’un index secondaire

```
CREATE INDEX ON table (nomcolonne);
```

    - L’index s’appelle généralement **table_nomcolonne_idx.**

- ## Pour supprimer l’index

```
DROP  INDEX table_nomcolonne_idx
```

- Il n’est pas du tout recommandé de construire un index lorsque:
  - Les données sont extrêmement différentes (ex: une adress email)
  - Les données sont très peu différentes (ex: une année)
  - Il vaut mieux dénormaliser le schéma, construire une autre table ayant une clé primaire adaptée.

- **Ajouter une colonne**
  - **Syntaxe:** ALTER TABLE <table> ADD <column type> ;
  - **Exemple** ALTER TABLE livre ADD auteur VARCHAR ;
- **Supprimer une colonne**
  - **Syntaxe** ALTER TABLE <table> DROP <column type> ;
  - **Exemple** ALTER TABLE livre DROP auteur ;
- **Supprimer une column family**
  - **Syntaxe** DROP COLUMNFAMILY <table> ;
  - **Exemple** DROP COLUMNFAMILY livre ;

- **Inserer une ligne**
  - **Syntaxe:**

```
INSERT INTO <table>(<colonnes....>) VALUES (<Valeurs...>);
INSERT INTO nomtable      JSON 'donnéesjson';
```

  - **Important:** La clé primaire doit obligatoirement être insérée

  - Contrairement à SQL, les noms des colonnes concernées sont obligatoires, mais toutes les colonnes n'ont pas obligation d'y être, les absentes seront affectées avec null.
  - **Exemple**
    - INSERT INTO clients (idclient, departement, nom) VALUES (1, 22,'Ali');
    - INSERT INTO clients (idclient, nom) VALUES (2, 'Salim');
    - INSERT INTO clients JSON '{"id":3,"nom":"Karim"}';

- **Insérer une ligne**
  - **Syntaxe:**

```
INSERT INTO <table>(<colonnes....>) VALUES (<Valeurs...>);
INSERT INTO nomtable      JSON 'donnéesjson';
```

  - **Important:** La clé primaire doit obligatoirement être insérée
- **Insertion avec une clé primaire naturelle**
  - **Exemple:**
  - INSERT INTO livre (isbn, titre) VALUES ('12548-5698', 'Le joueur d'échec');

- **Insertion avec une clé primaire composite**
  - **Exemple**
    - INSERT INTO livre (id, isbn, titre) VALUES (uuid(), '12548-5698', 'Le joueur d'échec') ;
  - On peut aussi utiliser des timeuuid, dont le calcul est basé sur l'heure.
    - INSERT INTO livre (id, isbn, titre) VALUES (now(), '12548-5698', 'Le joueur d'échec');

- Important: Les opérations de **delete** et **update** doivent être faites grâce à la clé primaire.
- **Supprimer une ligne**
  - **Syntaxe** **DELETE FROM** colonne **WHERE** <key> = <value>;
  - **Exemple** **DELETE FROM** livre **WHERE** isbn = '1258-5874’;

- **Modifier une ligne**
  - **Syntaxe** **UPDATE** <table> **SET** colonne = valeur **WHERE** <key> = <value>;
  - **Exemple:** **UPDATE** livre **SET** titre = 'Le rouge et le noir' **WHERE** isbn = '1258-5874';

- **Effectuer une requête**
  - **Syntaxe:**

```
SELECT  nomscolonnes...
FROM   table
[WHERE   condition]
[LIMIT  nombre]
[ALLOW  FILTERING];
```

- **Important :**
  - L'opération **SELECT** ne s'effectue que sur les colonnes clés, ou bien sur les colonnes indexées.
  - Il y a une très forte limite sur la clause **WHERE**:
    - Elle doit sélectionner des n-uplets contigus dans un index.
    - Donc c’est limité aux conditions sur les clés primaires ou secondaires.
    - On peut ajouter les mots-clés **ALLOW FILTERING** pour rompre cette contrainte mais ce n’est pas recommandé car ça oblige à traiter tous les n-uplets.

Création d'une base de données

- **Création d'un Keyspace**

CREATE KEYSPACE IF NOT EXISTS **DB_Ventes**
WITH REPLICATION = { 'class' : 'SimpleStrategy', 'replication_factor': 1};

- **L'utilisation de la BD: se positionner dans un keyspace**

USE BD_Ventes ;

- **Afficher les tables de cette BD**

DESC tables ;

C:\Windows\System32\cmd.exe - cqlsh 127.0.0.1 9042

```
cqlsh:bd_ventes> use bd_ventes
                 ... ;
cqlsh:bd_ventes> desc tables;

client  ventes

cqlsh:bd_ventes>
```

Modélisation des données

- **Normalisation vers la dénomalisation**
  - Classement par objets → requêtes / interrogation
  - Logique métier → logique applicative
  - Liens forts entre eux → pas de liens (jointure)

- **Unicité vers la duplication**
  - Table de référence / clés étrangère → pas de lien
  - Plusieurs tables peuvent stocker les mêmes informations réparties

Imbrication

- **Du fait de données distribuées, une jointure n'est pas envisageable.**
- **Il est donc préférable de regrouper les données le plus possible dans des lignes.**
  - Conception orientée sur les requêtes les plus fréquentes de l'application
  - Souvent favoriser la dimension la plus grande (meilleure distribution)
  - Le nouveau schéma génère de la redondance et des possibilités d'incohérence

- **Au pire, on représente les mêmes données sous plusieurs formes (vues matérialisées).**
  - Tous les vols, avec leurs hôtesses (solution envisageable)
  - A chaque hôtesse, la liste de ses vols

Imbrication

- **Collections**
  - Dans Cassandra, les collections permettent de stocker plusieurs valeurs dans uns seule variable.

- **3 types de collections**
  - **Set:** stocker un groupe d'éléments (ensemble de valeurs non ordonnées) qui seront restitué dans un ordre (alphanumérique, ...)
  - **List:** pour des éléments dont l'ordre importe ou si on doit stocker la même valeur plusieurs fois → (ensemble de valeurs ordonnées)
  - **Map:** stocker des paires d'éléments <clé, valeur>
  - **TYPE:** ligne imbriquée
  - **Tuple:** ligne imbriquée sans clés

- **Attention :**
  - Aucune collection ne peut contenir des éléments de plus de 64Ko car les collections ne sont pas distribuées puisqu'elles sont dans une même colonne.
  - Un set, une list ou une map n'existent que si il y a au moins un élément dedans. Donc le set vide est équivalent à la valeur nulle.

Exemple

- **CREATE TABLE** Vols (idVol INT, ligneID INT, dateDepart DATE,
  distance INT, duree FLOAT, pilote INT,
  copilote INT, officier INT, ChefCabine1 INT, ChefCabine2 INT,
  hotesses **XXX** ,
  aeroports **XXX** ,
  primary key (idVol));

Imbrication : Sets

- **Sets:**
  - Un ensemble (set) non ordonné de valeurs.
  - En utilisant le type de données **SET**, il est possible de résoudre le problème de champs multiples, comme les emails par exemple.

- **Création:** l'attribut multiple est représenté par un seule champ de type **set<TYPE>**
- **Par exemple :** set <INT>
  - **Exemple:** Liste des email d'un utilisateur

```
CREATE TABLE Utilisateurs ( user_id INT   PRIMARY KEY, Fname   TEXT, lname   TEXT ,
                                emails set<text>);
```

Imbrication : Sets

- **Insertion:**
  - pour insérer les donner dans l'ensemble, placer les valeurs entre accolades et les séparer par des virgules.
  - Les valeurs dans un ensemble doivent être uniques.
  - **Exemple:**

```
INSERT INTO Utilisateurs ( user_id, Fname, lname, emails)  VALUES
                (1234,c 'FRIHI', 'Ibtissem',{'ifrifi@usthb.univ', 'frihi@gmail.com'});
```

```
cqlsh:compagnie> select * from utilisateurs;

 user_id | emails                                        | fname | lname
---------+-----------------------------------------------+-------+----------
    1234 | {'frihi@gmail.com', 'ifrifi@usthb.univ'} | FRIHI | Ibtissem
```

Imbrication : Sets

- **Ajout:** Ajouter un élément à un ensemble en utilisant la commande UPDATE

```
UPDATE Utilisateurs
SET  emails =  {'newemails.univ'}
WHERE user_id =1234;
```

- **Sélection:** Les données sélectionnées sont retournées par ordre, selon le type. Les données textuelles sont ordonnées par ordre alphabétique

```
SELECT uset_id, emails, FROM utilisateurs WHERE user_id=1234
```

Imbrication : Sets

- **Suppression:** pour supprimer un élément de l'ensemble, utilisée l'operateur (-)

```
UPDATE Utilisateurs
SET  emails = emails - {'newemails.univ'}
WHERE user_id =1234;
```

- Pour supprimer tout l'ensemble

```
UPDATE Utilisateurs
SET  emails  = { }
WHERE user_id =1234;
```

- Ou bien

```
DELETE Emails
FROM Utilisateurs
WHERE user_id =1234;
```

Imbrication : Listes

- **Listes:**
  - est utilisée quand l'ordre d'insertion des éléments compte , ou quand on veut pouvoir insérer la même valeur plusieurs fois.
  - **Création:** liste de valeurs (ordonnées) de type **liste<TYPE>**
  - **Exemple:** Liste des meilleurs endroits visités

```
ALTER TABLE Utilisateurs ADD  top_places list<text>
```

  - **Insertion:** les éléments de la liste sont placées entre crochets
  - **Exemple:**

```
UPDATE Utilisateurs
SET top_places =['Tamanrasset','Asskrem']
 WHERE id_user=1234;
```

Imbrication : Listes

- **Ajouter:** permet d'ajouter un élément à la fin de la liste
- **Exemple:**

```
UPDATE Utilisateurs
SET  top_places = top_places + [' Djenet ' ]
WHERE   user_id=1234;
```

- Pour affecter un élément à un emplacement donnée et écraser l'ancienne valeur:
- **Exemple:**

```
UPDATE Utilisateurs
SET top_places [1] =' Djenet '
 WHERE user_id=1234;
```

Imbrication : Listes

- **Suppression:** permet de supprimer un élément de la liste en utilisant un index
- **Exemple:**

```
DELETE  top_places [3]
 FROM Utilisateurs
 WHERE   user_id=1234;
```

- La commande suivante protège contre les accès concurrents
- **Exemple:**

```
UPDATE Utilisateurs
SET top_places= top_places - ['Asskrem']
 WHERE user_id=1234;
```

Imbrication : Listes

- **Extraction:**
  - **Exemple:**

```
SELECT  user_id , top_places
FROM Utilisateurs
WHERE   user_id=1234;
```

Imbrication : Maps

- **Maps:**
  - il permet d'associer deux éléments, sous forme de cle/valeur.
  - Elle peut être utiliser pour sauvegarder les horaires des différents évènements dans un profil utilisateur par exemple.
  - **Création :**

```
ALTER table Utilisateurs  ADD  ToDo map<timestamp, text>
```

Imbrication : Maps

– **Insertion:** l'élément à ajouter doit être mis entre accolades séparé par une virgule.

- **Exemple:**

```
UPDATE Utilisateurs
SET  ToDo = {' 2023-9-24' : 'Biskra',
                    '2022-12-31': 'Taghit'}
WHERE   user_id=1234;
```

C:\Windows\System32\cmd.exe - cqlsh

```
cqlsh:bd_ventes> ALTER table Utilisateurs  ADD  ToDo map<timestamp, text>
             ... ;
cqlsh:bd_ventes> select * from utilisateurs;

 user_id | emails              | fname | lname    | todo | top_places
---------+---------------------+-------+----------+------+----------------------------
    1234 | {'newemails.univ'} | FRIHI | Ibtissem | null | ['Tamanrasset', ' Djenet ']

(1 rows)
cqlsh:bd_ventes> UPDATE Utilisateurs
             ...  SET  ToDo = {' 2023-9-24' : 'Biskra',
             ...                        '2022-12-31': 'Taghit'}
             ...  WHERE   user_id=1234;
cqlsh:bd_ventes>
```

- **Insertion:** l'élément à ajouter doit être mis entre accolades séparé par une virgule.

```
UPDATE Utilisateurs
SET  ToDo = ToDo+  {' 2023-9-24' : 'Biskra',
                    '2022-12-31': 'Taghit'}
WHERE   user_id=1234;
```

```
cqlsh:bd_ventes> select user_id, todo from utilisateurs;

 user_id | todo
---------+-----------------------------------------------------------------------------------------
    1234 | {'2022-12-30 23:00:00.000000+0000': 'Taghit', '2023-09-23 22:00:00.000000+0000': 'Biskra'}

(1 rows)
cqlsh:bd_ventes>
```

Imbrication : Maps

- **Ajout :** ajouter un élément spécifique en utilisant comme clé la valeur du timestamp entre crochets

```
UPDATE Utilisateurs
 SET  ToDo [' 2023-9-24  08:00'] = 'Timimoune'
WHERE   user_id=1234;
```

Ou bien

```
INSERT INTO Utilisateurs (user_id, ToDo)
 VALUES (1234, {' 2023-9-24  08:00': 'Biskra' });
```

Imbrication : Maps

- **Maps:** il permet d'associer deux éléments, sous forme de cle/valeur. Elle peut etre utiliser pour sauvegarder les horaires des différents évènements dans un profil utilisateur par exemple.
- **Suppression :**

```
DELETE ToDo [' 2023-9-24  08:00']
FROM Utilisateurs
WHERE   user_id=1234;
```

- **Extraction :**

```
SELECT user_id , ToDo
FROM Utilisateurs
WHERE   user_id=1234;
```

Imbrication : Type

- **TYPE:** Il faut figer le type (frozen <newType>
  - Le mot-clé "frozen" est utilisé pour déclarer des types de données composites, tels que les types de données définis par l'utilisateur (UDT) ou les collections imbriquées (listes, ensembles, maps).
- **Création :**

```
Create TYPE fullname ( fname   text, lname   text):
Create TYPE address ( City   text, nation  text, region text);
```

```
CREATE TABLE users
( id int  PRIMARY KEY,
  name frozen <fullname>,
  direct_reports set<frozen <fullname>>,
  addresses map<text, frozen <address>>);
```

- **Insertion:** Insertion d'un nouvel utilisateur avec des données complexes

```
INSERT INTO users ( id, name, direct_reports, addresses )
VALUES
 ( 1,
  {first_name: 'John', last_name: 'Doe’},
  {     {first_name: 'Alice', last_name: 'Smith’},
        {first_name: 'Bob', last_name: 'Johnson’}
  },
 {'home': {street: '123 Main St', city: 'Cityville', zip_code: '12345’},
   'work': {street: '456 Business St', city: 'Worktown', zip_code: '54321’}
 }
);
```

- **UPDATE:** mettre a jours un champs frozen
  - Pour mettre à jour un champ de type "frozen" dans Cassandra, vous devez effectuer une opération d'insertion complète en utilisant la clé primaire de la ligne que vous souhaitez mettre à jour.
  - Lorsque vous mettez à jour un champ "frozen", vous devez réinsérer l'intégralité de la structure complexe, car Cassandra ne prend pas en charge les mises à jour partielles de champs congelés.
- **Exemple:**

```
UPDATE keyspace_name.users
SET name = {first_name: 'NouveauPrénom', last_name: 'NouveauNom’},
    direct_reports = {
                        {first_name: 'NouvelAlice', last_name: 'NouvelleSmith’},
                        {first_name: 'NouveauBob', last_name: 'NouveauJohnson’}},
    addresses = {'nouvelleMaison': {street: '456 NouvelleRue', city: 'NouvelleVille', zip_code: '67890’}}
WHERE id = 1;
```